

AI-Assisted Software Testing

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This syllabus lists every module of the course in teaching order, with the topics covered in each. It is generated from the course page, so the two cannot disagree.

Module 1: Generative AI for QA Professionals

Schedule: Week 1

The mental model first, kept at exactly the depth a tester needs. AI, machine learning and Generative AI as distinct things; what a large language model actually does; prompts, system and user instructions, tokens and context.

Then the properties that matter operationally: the same prompt can give different answers, the model will state something false with complete confidence, and it has no way to signal uncertainty reliably. Human-in-the-loop is introduced here as the governing principle for everything that follows — the tester approves, always.

Topics

- AI vs machine learning vs Generative AI
- What a large language model is, in tester's terms
- Prompts, system instructions and user instructions
- Tokens and context, and why long chats drift
- Model variability — why the same prompt differs twice
- Hallucination: confident, fluent and wrong
- Structured output and why it matters for QA
- The human-in-the-loop principle

Module 2: Prompt Engineering for Testing

Schedule: Week 1

Prompting as a QA skill rather than a general one. A prompt that produces good test cases contains the requirement, the constraints, the acceptance criteria and the output format — and most weak AI output traces to a prompt missing one of those.

You build reusable prompt templates for the tasks you repeat every sprint, and learn role prompting and iterative refinement. The module ends on evaluation: how to read an AI answer critically before you use it, which is the habit that separates the two groups of adopters.

Topics

- Clear task definition and stated constraints
- Supplying context — how much, and what to leave out
- Acceptance criteria inside the prompt
- Examples and few-shot patterns
- Output schemas and format control
- Role prompting for QA tasks
- Iterative refinement without starting over
- Building a reusable QA prompt template
- Evaluating an AI answer before you use it

Module 3: AI-Assisted Requirement Analysis

Schedule: Week 1

The highest-leverage use, and the least obvious one. Testers find defects in requirements more cheaply than anyone finds them in code, and an assistant is genuinely good at surfacing ambiguity, contradiction and omission in a specification.

You practise on real, imperfect requirements: summarising them, listing what is undefined, spotting rules that contradict each other, generating the clarification questions to take to the BA, and deriving acceptance criteria. Comparing two versions of a specification to find what silently changed is included, because that is where regressions are born.

Topics

- Summarising a specification without losing detail
- Identifying ambiguity and undefined behaviour
- Finding missing rules and edge conditions
- Detecting contradictions between requirements
- Generating clarification questions for the BA
- Deriving acceptance criteria from prose
- Producing a risk list from a specification
- Tracing requirements to tests
- Diffing two versions of a requirement

Module 4: AI-Assisted Test Design

Schedule: Week 2

Generating test scenarios and cases, and — the part that carries the module — deciding which ones to keep. An assistant will readily produce forty cases, of which perhaps twenty-five are genuine, ten are duplicates in different words and five test something the system does not do.

The classical techniques are used as the check: if the generated set has no boundary values, no negative cases and no state transitions, it is incomplete regardless of how long it is. Risk-based prioritisation and regression selection follow, because the constraint is never how many cases you can write.

Topics

- Generating test scenarios from a requirement
- Functional, negative and boundary cases
- Decision tables and state-transition cases with AI
- Exploratory charters generated as starting points
- Spotting duplicates and non-existent behaviour
- Using classical techniques to audit the generated set
- Risk-based prioritisation
- Regression selection for a release
- Cross-browser, device and accessibility test ideas
- API test scenarios from a contract

Module 5: AI-Assisted Test Data

Schedule: Week 2

Test data is where AI saves the most unglamorous hours. Synthetic datasets, positive and negative sets, boundary sets, JSON payloads, CSV files and SQL seed scripts — generated in minutes rather than an afternoon of typing.

The discipline is privacy. Pasting a production extract into a public assistant to "make more like this" is a data-protection incident, and it happens. You learn masking, synthetic generation from a schema rather than from real records, and how to get genuinely diverse data — names, scripts, address formats — rather than fifty rows of near-identical filler.

Topics

- Synthetic data generated from a schema
- Positive, negative and boundary datasets
- JSON payloads and CSV files

- SQL seed scripts
- Masking and de-identification
- Why production extracts must never be pasted in
- PII considerations and what your policy likely says
- Getting real diversity, not fifty similar rows
- Validating generated data against the schema

Module 6: AI-Assisted Automation

Schedule: Week 3

Drafting automation code with AI across the stacks this category teaches — Selenium with Java or Python, Playwright with TypeScript, API assertions, pytest and TestNG. Generating locators, converting manual cases into automation candidates, producing Page Objects, and explaining unfamiliar legacy test code.

Reviewing generated test code closes the module and is weighted accordingly. Automation code that passes for the wrong reason is worse than no automation, because it creates confidence without coverage — and generated tests fail this way more often than hand-written ones.

Topics

- Selenium test skeletons in Java and Python
- Playwright tests in TypeScript
- API assertions and response validation
- pytest and TestNG test generation
- Locator candidates and their trade-offs
- Converting manual cases to automation candidates
- Page Object generation and refactoring
- Explaining legacy test code you inherited
- Test maintenance with AI
- Reviewing generated test code — passing for the wrong reason

Module 7: AI-Assisted Failure Analysis

Schedule: Week 3

Triage, which is where testers lose the most time to the least interesting work. Interpreting stack traces, explaining application logs, and narrowing a failure to a likely cause — with AI as a fast first reader rather than the decision-maker.

The judgement being built is the distinction between a test failure and an application failure, because reporting the wrong one wastes a developer's day and your credibility. Flaky-test investigation, locator repair, screenshot comparison and drafting a reproducible defect report from evidence complete the module.

Topics

- Interpreting a stack trace
- Explaining application and server logs
- Narrowing to a likely root cause
- Test failure versus application failure
- Flaky-test investigation patterns
- Locator repair after a UI change
- Analysing screenshots and visual diffs
- Comparing expected and actual API responses
- Drafting a reproducible defect report from evidence

Module 8: QA Documentation and Team Workflow

Schedule: Week 4

The written output QA owes everyone else. Test plans, test summary reports, defect descriptions, release notes, traceability matrices and status updates — all of them tasks where a good first draft in ninety seconds changes what actually gets written.

Then the team layer: AI-assisted code review, AI in pull requests, and building a shared prompt library so the whole QA team benefits rather than one enthusiast. Knowledge-base creation from scattered documents is included, because most QA teams have that problem and few solve it.

Topics

- Test plans and test summary reports
- Defect descriptions that a developer can act on
- Release notes from a defect list
- Traceability matrices
- Status updates and stakeholder summaries
- AI-assisted code review
- AI in pull requests
- Building a shared QA prompt library
- Knowledge-base creation from scattered documents

Module 9: Responsible AI for QA

Schedule: Week 4

The module that keeps the rest of the course usable at work. Data privacy and PII, credentials and secrets, proprietary code and the licensing questions around generated code — each with the practical version of the rule rather than the legal abstraction.

Over-reliance is treated as a real risk rather than a caution: a tester who stops reading is worse than a tester who never started prompting. Prompt-injection awareness, human approval gates and aligning with enterprise AI policy close the course, so you can take this back to a team that has rules.

Topics

- Data privacy and PII in prompts
- Credentials and secrets — the rule and why it is absolute
- Proprietary code and what leaves your network
- Licensing questions around generated code
- Hallucination risk in QA artefacts specifically
- Over-reliance, and the tester who stopped reading
- Prompt-injection awareness
- Human approval gates
- Aligning with enterprise AI policy

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.